Name: Mrigaj Shaw
Roll No: 2024CSB041

Assignment 7

Problem 1

Problem Statement: In this assignment, you will design and implement a smart navigation system for a city with 10 locations (vertices) such as schools, hospitals, and markets. The roads between these locations are represented as edges:

$$E = \{(u, v, w) | w_i > 0\}$$

where each edge has a positive travel cost (distance or time). Dynamically construct the directed graph and compute the shortest path from a selected source location using two different approaches: adjacency list representation and adjacency matrix representation.

Your task:

1. **Graph Construction**
   - Define the vertex set: $V = \{v_1, v_2, ..., v_{10}\}$
   - Dynamically construct the graph by allowing the user to:
     - Select any two vertices (u, v)
     - Assign a positive weight $w_i > 0$
   - Represent the same graph in two formats:
     - Adjacency List Representation
     - Adjacency Matrix Representation
2. **Scenario 1: Adjacency List + Min-Heap**
   - Implement Dijkstra's Algorithm using adjacency list
   - Use a min-heap (priority queue) to select the next vertex with minimum distance
   - Compute the shortest path from a chosen source vertex to all other vertices
3. **Scenario 2: Adjacency Matrix + Min-Heap**
   - Implement Dijkstra's Algorithm using adjacency matrix
   - Use a min-heap for selecting the next vertex
   - Compute the shortest path from the same source vertex
4. **Performance Evaluation**
   - Execute both implementations on the same graph input and measure:
     - Execution time t
   - Perform analysis under:
     - Sparse Graph (few edges)
     - Dense Graph (many edges)

Code

```c
#include "../../Assignment_6/AdjacencyList/AdjacencyList.h"
#include "../../Assignment_6/MinHeap/MinHeap.h"
#include <limits.h>
#include <stdio.h>
#include <time.h>

void dijkstraList(Node **adjList, int vertices, int startNode) {
    int dist[vertices];
    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
    }
    dist[startNode] = 0;

    MinHeap *pq = createMinHeap(200);
    insertMinHeap(pq, -1, startNode, 0);

    while (!isEmpty(pq)) {
        Edge current = extractMin(pq);
        int u = current.v;
        int d = current.weight;

        if (d > dist[u])
            continue;

        Node *temp = adjList[u];
        while (temp != NULL) {
            int v = temp->vertex;
            int weight = temp->weight;

            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                insertMinHeap(pq, u, v, dist[v]);
            }
            temp = temp->next;
        }
    }

    printf("\n[List] Shortest Distances from Source %d:\n", startNode);
    for (int i = 0; i < vertices; i++) {
        if (dist[i] == INT_MAX)
            printf("Vertex %d: Unreachable\n", i);
        else
            printf("Vertex %d: %d\n", i, dist[i]);
    }
    freeMinHeap(pq);
}

void dijkstraMatrix(int vertices, int matrix[vertices][vertices],
                    int startNode) {
    int dist[vertices];
    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
    }
    dist[startNode] = 0;

    MinHeap *pq = createMinHeap(200);
    insertMinHeap(pq, -1, startNode, 0);

```

```c
    while (!isEmpty(pq)) {
        Edge current = extractMin(pq);
        int u = current.v;
        int d = current.weight;

        if (d > dist[u])
            continue;

        for (int v = 0; v < vertices; v++) {
            if (matrix[u][v] != INT_MAX) {
                int weight = matrix[u][v];
                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    insertMinHeap(pq, u, v, dist[v]);
                }
            }
        }
    }

    printf("\n[Matrix] Shortest Distances from Source %d:\n", startNode);
    for (int i = 0; i < vertices; i++) {
        if (dist[i] == INT_MAX)
            printf("Vertex %d: Unreachable\n", i);
        else
            printf("Vertex %d: %d\n", i, dist[i]);
    }
    freeMinHeap(pq);
}

int main() {
    int vertices = 10;
    int edges = 0;
    int u, v, w;
    clock_t start_time, end_time;
    double time_list, time_matrix;

    Node **AdjacencyList = createAdjacencyList(vertices);
    if (!AdjacencyList) {
        printf("[ERROR] Adjacency list initialization failed\n");
        return -1;
    }

    int AdjacencyMatrix[vertices][vertices];
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (i == j) {
                AdjacencyMatrix[i][j] = 0;
            } else {
                AdjacencyMatrix[i][j] = INT_MAX;
            }
        }
    }

    printf("Enter number of edges (max 90): ");
    if (scanf("%d", &edges) != 1 || edges > 90) {
        printf("Invalid edge count.\n");
        return -1;
    }

    printf("Enter edges (u v w) where u, v < 10:\n");
```

```c
    for (int i = 0; i < edges; i++) {
        if (scanf("%d %d %d", &u, &v, &w) != 3)
            break;
        if (u >= vertices || v >= vertices || u < 0 || v < 0 || w < 0) {
            printf("Invalid vertex index/negative weight! Skipping edge.\n");
            continue;
        }

        addEdgeAdjacencyListDirectedWithWeight(AdjacencyList, u, v, w);
        AdjacencyMatrix[u][v] = w;
    }

    displayAdjacencyList(AdjacencyList, vertices);

    int startNode;
    printf("Enter source vertex for Dijkstra: ");
    scanf("%d", &startNode);

    if (startNode >= 0 && startNode < vertices) {

        start_time = clock();
        dijkstraList(AdjacencyList, vertices, startNode);
        end_time = clock();
        time_list = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;

        start_time = clock();
        dijkstraMatrix(vertices, AdjacencyMatrix, startNode);
        end_time = clock();
        time_matrix = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;

        printf("\nPerformance Evaluation\n");
        printf("Adjacency List Execution Time: %f seconds\n", time_list);
        printf("Adjacency Matrix Execution Time: %f seconds\n", time_matrix);

    } else {
        printf("Invalid source vertex.\n");
    }

    freeAdjacencyList(AdjacencyList, vertices);
    return 0;
}
```

Output

```
Enter number of edges (max 90): 12
Enter edges (u v w) where u, v < 10:
0 1 4
0 2 2
1 3 5
2 1 1
2 3 8
2 4 10
3 5 2
4 5 6
5 6 3
6 7 4
7 8 1
8 9 2

Adjacency List Representation
-------------------------------
Vertex [0]: 2(w:2) -> 1(w:4) -> NULL
Vertex [1]: 3(w:5) -> NULL
Vertex [2]: 4(w:10) -> 3(w:8) -> 1(w:1) -> NULL
Vertex [3]: 5(w:2) -> NULL
Vertex [4]: 5(w:6) -> NULL
Vertex [5]: 6(w:3) -> NULL
Vertex [6]: 7(w:4) -> NULL
Vertex [7]: 8(w:1) -> NULL
Vertex [8]: 9(w:2) -> NULL
Vertex [9]: NULL
```

```
Enter source vertex for Dijkstra: 0

[List] Shortest Distances from Source 0:
Vertex 0: 0
Vertex 1: 3
Vertex 2: 2
Vertex 3: 8
Vertex 4: 12
Vertex 5: 10
Vertex 6: 13
Vertex 7: 17
Vertex 8: 18
Vertex 9: 20

[Matrix] Shortest Distances from Source 0:
Vertex 0: 0
Vertex 1: 3
Vertex 2: 2
Vertex 3: 8
Vertex 4: 12
Vertex 5: 10
Vertex 6: 13
Vertex 7: 17
Vertex 8: 18
Vertex 9: 20
```

```
Performance Evaluation
Adjacency List Execution Time: 0.000000 seconds
Adjacency Matrix Execution Time: 0.000000 seconds
```

```
Enter number of edges (max 90): 77
Enter edges (u v w) where u, v < 10:
0 1 5
0 2 3
0 3 8
0 4 2
0 5 7
0 6 1
0 7 9
0 8 4
1 0 2
1 2 6
1 3 1
1 4 8
1 5 3
1 7 5
1 9 4
2 0 7
2 1 4
2 3 2
2 5 1
2 6 9
2 8 5
2 9 3
```

```
Adjacency List Representation
-----------------------------
Vertex [0]: 8(w:4) -> 7(w:9) -> 6(w:1) -> 5(w:7) -> 4(w:2) -> 3(w:8) -> 2(w:3) -> 1(w:5) -> NULL
Vertex [1]: 9(w:4) -> 7(w:5) -> 5(w:3) -> 4(w:8) -> 3(w:1) -> 2(w:6) -> 0(w:2) -> NULL
Vertex [2]: 9(w:3) -> 8(w:5) -> 6(w:9) -> 5(w:1) -> 3(w:2) -> 1(w:4) -> 0(w:7) -> NULL
Vertex [3]: 9(w:4) -> 8(w:7) -> 7(w:1) -> 6(w:2) -> 5(w:6) -> 4(w:5) -> 2(w:8) -> 1(w:3) -> NULL
Vertex [4]: 8(w:2) -> 7(w:8) -> 6(w:3) -> 5(w:4) -> 3(w:6) -> 2(w:5) -> 0(w:1) -> NULL
Vertex [5]: 9(w:6) -> 8(w:9) -> 7(w:3) -> 6(w:7) -> 4(w:1) -> 3(w:4) -> 1(w:2) -> 0(w:8) -> NULL
Vertex [6]: 9(w:7) -> 8(w:1) -> 7(w:4) -> 5(w:8) -> 4(w:2) -> 3(w:9) -> 2(w:3) -> 1(w:5) -> NULL
Vertex [7]: 9(w:2) -> 8(w:6) -> 6(w:3) -> 5(w:5) -> 4(w:9) -> 3(w:1) -> 2(w:7) -> 0(w:4) -> NULL
Vertex [8]: 9(w:4) -> 7(w:5) -> 6(w:8) -> 5(w:1) -> 4(w:7) -> 2(w:2) -> 1(w:9) -> 0(w:3) -> NULL
Vertex [9]: 8(w:7) -> 7(w:9) -> 6(w:2) -> 4(w:3) -> 3(w:5) -> 2(w:8) -> 1(w:1) -> 0(w:6) -> NULL
```

```
Enter source vertex for Dijkstra: 0

[List] Shortest Distances from Source 0:
Vertex 0: 0
Vertex 1: 5
Vertex 2: 3
Vertex 3: 5
Vertex 4: 2
Vertex 5: 3
Vertex 6: 1
Vertex 7: 5
Vertex 8: 2
Vertex 9: 6

[Matrix] Shortest Distances from Source 0:
Vertex 0: 0
Vertex 1: 5
Vertex 2: 3
Vertex 3: 5
Vertex 4: 2
Vertex 5: 3
Vertex 6: 1
Vertex 7: 5
Vertex 8: 2
Vertex 9: 6
```

```
Performance Evaluation
Adjacency List Execution Time: 0.001000 seconds
Adjacency Matrix Execution Time: 0.000000 seconds
```

Inference

- **Time Complexity (Adjacency List):** $O((V + E) \log V)$. This approach is highly efficient for extracting the minimum distance node and iterating only through the existing neighbors.
- **Time Complexity (Adjacency Matrix):** $O(V^2 + E \log V)$. Even with a min-heap, the matrix requires iterating over all V possible vertices to find connected neighbors, making it strictly bounded by $O(V^2)$.
- **Sparse vs. Dense Graph Performance:** For sparse graphs ($E \approx V$), the adjacency list implementation executes significantly faster and consumes less memory. For dense graphs ($E \approx V^2$), the adjacency matrix implementation performs competitively, though the list representation remains mathematically optimal.

Problem 2

Problem Statement: In this assignment, you will explore an important limitation of Dijkstra's Algorithm by designing a graph with $V = 10$ vertices where the user dynamically constructs edges $E = \{(u, v, w)\}$, including the possibility of negative edge weights and even a negative weight cycle. While Dijkstra's Algorithm assumes $w_i > 0$, you will intentionally violate this condition by allowing $w_i < 0$ and analyze how the algorithm fails to compute correct shortest paths from a source vertex s.

Dijkstra relies on the greedy assumption that once a node u is selected with minimum distance, its shortest path is finalized. However, in the presence of negative edges, this assumption breaks:

$$\exists (u, v) \in E \text{ such that } w(u, v) < 0 \Rightarrow dist[v] < dist[u] + w(u, v)$$

even after node u has already been processed.

Negative Cycle Condition

If a negative cycle exists such that $\sum_{(u,v)\in C} w(u, v) < 0$, then the shortest path becomes undefined because distances can decrease indefinitely: $dist[v] \to -\infty$.

Your task:

- Dynamically create a graph with 10 vertices
- Add edges between vertices, including negative weights
- Intentionally create a cycle with total negative weight
- Apply Dijkstra's Algorithm from a source vertex
- Observe incorrect results or failure to detect the negative cycle

Code

```c
#include "../../Assignment_6/AdjacencyList/AdjacencyList.h"
#include "../../Assignment_6/MinHeap/MinHeap.h"
#include <limits.h>
#include <stdio.h>

void dijkstraList(Node **adjList, int vertices, int startNode) {
    int dist[vertices];
    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
    }
    dist[startNode] = 0;

    MinHeap *pq = createMinHeap(1000);
    insertMinHeap(pq, -1, startNode, 0);

    int extraction_count = 0;
    int max_safe_extractions = 200; // Circuit breaker to prevent infinite loops

    printf("\nStarting Dijkstra from vertex %d...\n", startNode);

    while (!isEmpty(pq)) {
        Edge current = extractMin(pq);
        int u = current.v;
        int d = current.weight;

        extraction_count++;

        if (extraction_count > max_safe_extractions) {
            printf("\n[ABORT] Infinite loop detected!\n");
            printf("Extraction count exceeded %d. Negative cycle exists.\n",
                   max_safe_extractions);
            break;
        }

        if (d > dist[u])
            continue;

        Node *temp = adjList[u];
        while (temp != NULL) {
            int v = temp->vertex;
            int weight = temp->weight;

            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;

                // Print the update to observe the algorithm failing
                printf("Update: dist[%d] dropped to %d (via %d)\n", v, dist[v],
                       u);

                insertMinHeap(pq, u, v, dist[v]);
            }
            temp = temp->next;
        }
    }

    printf("\nFinal Distances (Likely Incorrect or Incomplete):\n");
    for (int i = 0; i < vertices; i++) {
        if (dist[i] == INT_MAX)
```

```
            printf("Vertex %d: Unreachable\n", i);
        else
            printf("Vertex %d: %d\n", i, dist[i]);
    }

    freeMinHeap(pq);
}

int main() {
    int vertices = 10;
    int edges = 0;
    int u, v, w;

    Node **AdjacencyList = createAdjacencyList(vertices);
    if (!AdjacencyList)
        return -1;

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    printf("Enter edges (u v w) - Negative weights allowed:\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d %d", &u, &v, &w);
        addEdgeAdjacencyListDirectedWithWeight(AdjacencyList, u, v, w);
    }

    int startNode;
    printf("Enter source vertex: ");
    scanf("%d", &startNode);

    if (startNode >= 0 && startNode < vertices) {
        dijkstraList(AdjacencyList, vertices, startNode);
    }

    freeAdjacencyList(AdjacencyList, vertices);
    return 0;
}
```

Output

```
Enter number of edges: 5
Enter edges (u v w) - Negative weights allowed:
0 1 5
1 2 2
2 3 -10
3 1 4
0 4 1
Enter source vertex: 0

Starting Dijkstra from vertex 0...
Update: dist[4] dropped to 1 (via 0)
Update: dist[1] dropped to 5 (via 0)
Update: dist[2] dropped to 7 (via 1)
Update: dist[3] dropped to -3 (via 2)
Update: dist[1] dropped to 1 (via 3)
Update: dist[2] dropped to 3 (via 1)
Update: dist[3] dropped to -7 (via 2)
Update: dist[1] dropped to -3 (via 3)
Update: dist[2] dropped to -1 (via 1)
Update: dist[3] dropped to -11 (via 2)
Update: dist[1] dropped to -7 (via 3)
Update: dist[2] dropped to -5 (via 1)
Update: dist[3] dropped to -15 (via 2)
Update: dist[1] dropped to -11 (via 3)
Update: dist[2] dropped to -9 (via 1)
```

```
[ABORT] Infinite loop detected!
Extraction count exceeded 200. Negative cycle exists.

Final Distances (Likely Incorrect or Incomplete):
Vertex 0: 0
Vertex 1: -259
Vertex 2: -253
Vertex 3: -263
Vertex 4: 1
Vertex 5: Unreachable
Vertex 6: Unreachable
Vertex 7: Unreachable
Vertex 8: Unreachable
Vertex 9: Unreachable
```

Inference

- **Algorithmic Failure:** Dijkstra's algorithm fundamentally relies on a greedy strategy, operating under the assumption that adding an edge to a path can only increase its total weight.
- **Negative Edges:** When a negative edge is introduced, the algorithm may finalize a node's distance too early. A later, longer path containing a negative edge might actually result in a cheaper overall cost, which Dijkstra will ignore since the target node was already popped from the priority queue.
- **Negative Cycles:** In the presence of a negative weight cycle, the concept of a "shortest path" mathematically breaks down, as traversing the cycle infinitely yields an infinitely negative cost. Dijkstra has no mechanism to detect this and outputs incorrect, static values based on its initial arbitrary traversals.

Problem 3

Problem Statement: You are given a set of n points: $S = \{p_1, p_2, ..., p_n\}$, where $p_i = (x_i, y_i)$. Your goal is to find the pair of points (p_i, p_j) that have the smallest distance among all pairs.

The assignment considers two cases:

Case 1 - One-Dimensional Points (1D)

Each point is represented as $p_i = (x_i)$. The distance between any two points is calculated as $d = |x_i - x_j|$. You are required to find the pair of points with the minimum distance in 1D.

Case 2 - Two-Dimensional Points (2D)

Each point is represented as $p_i = (x_i, y_i)$. The distance between any two points is calculated using the Euclidean formula $d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. You are required to find the pair of points with the minimum distance in 2D.

Your tasks

1. Implement a method to find the closest pair of points for 1D points.
2. Implement a method to find the closest pair of points for 2D points.
3. Compare the time complexity of the 1D and 2D implementations.

Code

```c
#include <float.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    double x, y;
} Point;

int compare1D(const void *a, const void *b) {
    double difference = (*(double *)a - *(double *)b);
    return (difference > 0) - (difference < 0);
}

double closestPair1D(double *array, int n) {
    qsort(array, n, sizeof(double), compare1D);
    double minDistance = DBL_MAX;

    for (int i = 0; i < n - 1; i++) {
        if (array[i + 1] - array[i] < minDistance)
            minDistance = array[i + 1] - array[i];
    }
    return minDistance;
}

int compareX(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    return (p1->x > p2->x) - (p1->x < p2->x);
}

int compareY(const void *a, const void *b) {
    Point *p1 = (Point *)a;
    Point *p2 = (Point *)b;

    return (p1->y > p2->y) - (p1->y < p2->y);
}

double distance2D(Point p1, Point p2) {
    return sqrt((p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y));
}

double minDouble(double x, double y) { return (x < y) ? x : y; }

double closestUtil(Point *Px, Point *Py, int n) {
    if (n <= 3) {
        double minVal = DBL_MAX;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                double distance = distance2D(Px[i], Py[i]);
                minVal = minDouble(distance, minVal);
            }
        }
        return minVal;
    }

```

```
    int mid = n / 2;
    Point midPoint = Px[mid];

    Point *PyL = (Point *)malloc(mid * sizeof(Point));
    Point *PyR = (Point *)malloc((n - mid) * sizeof(Point));

    int l = 0, r = 0;

    for (int i = 0; i < n; i++) {
        if ((Py[i].x < midPoint.x || (Py[i].x == midPoint.x && l < mid)) &&
            l < mid) {
            PyL[l++] = Py[i];
        } else
            PyR[r++] = Py[i];
    }

    double dl = closestUtil(Px, PyL, mid);
    double dr = closestUtil(Px + mid, PyR, n - mid);

    double d = minDouble(dl, dr);

    Point *strip = (Point *)malloc(n * sizeof(Point));

    int j = 0;
    for (int i = 0; i < n; i++) {
        if (fabs(Py[i].x - midPoint.x) < d)
            strip[j++] = Py[i];
    }

    double minVal = d;

    for (int i = 0; i < j; i++) {
        for (int k = i + 1; k < j && (strip[k].y - strip[i].y) < minVal; k++) {
            double distance = distance2D(strip[i], strip[k]);
            minVal = minDouble(minVal, distance);
        }
    }

    free(PyL);
    free(PyR);
    free(strip);

    return minVal;
}

double closestPair2D(Point *P, int n) {
    Point *Px = (Point *)malloc(n * sizeof(Point));
    Point *Py = (Point *)malloc(n * sizeof(Point));

    for (int i = 0; i < n; i++) {
        Px[i] = P[i];
        Py[i] = P[i];
    }

    qsort(Px, n, sizeof(Point), compareX);
    qsort(Py, n, sizeof(Point), compareY);

    double ans = closestUtil(Px, Py, n);

    free(Px);
```

```c
    free(Py);

    return ans;
}

int main() {
    int choice, n;

    printf("Select Case:\n1. 1D Points\n2. 2D Points\nEnter choice: ");
    if (scanf("%d", &choice) != 1)
        return 1;

    printf("Enter the number of points: ");
    if (scanf("%d", &n) != 1 || n < 2) {
        printf("Need at least 2 points.\n");
        return 1;
    }

    if (choice == 1) {
        double *arr = (double *)malloc(n * sizeof(double));
        printf("Enter %d points (X coordinates):\n", n);
        for (int i = 0; i < n; i++) {
            scanf("%lf", &arr[i]);
        }

        printf("The smallest distance is: %lf\n", closestPair1D(arr, n));
        free(arr);
    } else if (choice == 2) {
        Point *P = (Point *)malloc(n * sizeof(Point));
        printf("Enter %d points (X Y coordinates separated by space):\n", n);
        for (int i = 0; i < n; i++) {
            scanf("%lf %lf", &P[i].x, &P[i].y);
        }

        printf("The smallest distance is: %lf\n", closestPair2D(P, n));
        free(P);
    } else {
        printf("Invalid choice.\n");
    }

    return 0;
}
```

Output

```
Select Case:
1. 1D Points
2. 2D Points
Enter choice: 2
Enter the number of points: 2
Enter 2 points (X Y coordinates separated by space):
2 1
1 2
The smallest distance is: 1.414214
```

Inference

- **1D Time Complexity:** The 1D approach resolves in $O(n \log n)$ time. This is achieved by sorting the array of points ($O(n \log n)$) and then performing a single linear scan ($O(n)$) to compute the distance between adjacent elements.
- **2D Time Complexity:** The 2D approach uses a Divide and Conquer methodology, dropping the complexity from the naive $O(n^2)$ down to $O(n \log n)$.
- **Implementation Comparison:** While both algorithms share an $O(n \log n)$ time complexity bound, the 2D implementation carries a significantly larger constant factor. The 2D case requires maintaining multiple sub-arrays, recursive tree unwinding, and filtering a boundary "strip" width of $2d$ at every level of division, making it considerably heavier on memory and execution time overhead than the 1D linear scan.

Problem 4

Problem Statement: In this assignment, you are given a weighted directed graph $G = (V, E)$ where V is the set of vertices and E is the set of edges. Each edge $(u, v) \in E$ has a weight $w(u, v)$, which can be either positive or negative. Your task is to explore the behavior of the Bellman-Ford Algorithm when the graph contains a negative weight cycle.

The Bellman-Ford algorithm is used to find the shortest path from a source vertex s to all other vertices. The distance from s to any vertex v is calculated as:

$$d(v) = \min_{(u,v)\in E} (d(u) + w(u, v))$$

for all edges $(u, v) \in E$.

If the graph contains a negative weight cycle (i.e., a cycle whose total weight is negative): $\sum_{(u,v)\in cycle} w(u, v) < 0$, the Bellman-Ford algorithm can detect the presence of the cycle after $|V| - 1$ relaxations. This is done by checking if any edge can still reduce the distance: If $d(u) + w(u, v) < d(v)$ after $|V| - 1$ iterations, a negative weight cycle exists.

Your tasks

1. Implement the Bellman-Ford algorithm for a weighted directed graph.

2. Detect whether a negative weight cycle exists using the final relaxation check.

Code

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, weight;
} Edge;

void bellmanFord(Edge *edges, int V, int E, int source) {
    int *dist = (int *)malloc(V * sizeof(int));

    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
    }
    dist[source] = 0;

    for (int i = 1; i < V; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int weight = edges[j].weight;
            if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
            }
        }
    }

    for (int j = 0; j < E; j++) {
        int u = edges[j].u;
        int v = edges[j].v;
        int weight = edges[j].weight;

        if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
            printf("\nGraph contains a negative weight cycle! Shortest paths "
                   "are undefined.\n");
            free(dist);
            return;
        }
    }

    printf("\nVertex \t Distance from Source (%d)\n", source);
    for (int i = 0; i < V; i++) {
        if (dist[i] == INT_MAX) {
            printf("%d \t INF\n", i);
        } else {
            printf("%d \t %d\n", i, dist[i]);
        }
    }

    free(dist);
}

int main() {
    int V, E, source;

    printf("Enter number of vertices: ");
    if (scanf("%d", &V) != 1)
        return 1;
```

```c

    printf("Enter number of edges: ");
    if (scanf("%d", &E) != 1)
        return 1;

    Edge *edges = (Edge *)malloc(E * sizeof(Edge));

    printf("Enter %d edges (format: source destination weight):\n", E);
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    printf("Enter the source vertex: ");
    if (scanf("%d", &source) != 1)
        return 1;

    bellmanFord(edges, V, E, source);

    free(edges);

    return 0;
}
```

Output

```
Enter number of vertices: 4
Enter number of edges: 4
Enter 4 edges (format: source destination weight):
0 1 1
1 2 2
2 3 3
3 1 -6
Enter the source vertex: 0

Graph contains a negative weight cycle! Shortest paths are undefined.
```

Inference

- **Time Complexity:** The Bellman-Ford algorithm operates with a time complexity of $O(V \times E)$. This is because it must iterate over all E edges exactly $V - 1$ times to guarantee the shortest paths are found.
- **Handling Negative Edges:** Unlike Dijkstra's algorithm, Bellman-Ford correctly processes graphs with negative edge weights by continually relaxing edges, ensuring all potential path reductions are exhausted.
- **Cycle Detection:** By executing one additional (V-th) pass over all edges, the algorithm easily flags negative weight cycles. If the distance to any vertex is further minimized during this extra pass, it mathematically proves the existence of a negative cycle, allowing the program to safely halt and reject the undefined graph.

Appendix: Header Files

Note: Below are the custom header files utilized across the assignment implementations to maintain modularity.

MinHeap.h

```c
#ifndef MIN_HEAP_H
#define MIN_HEAP_H

#include <stdbool.h>
#include <stdlib.h>

// structure to hold graph edges for the priority queue
typedef struct {
    int u;      // source vertex
    int v;      // destination vertex
    int weight; // edge cost
} Edge;

typedef struct {
    Edge *array;
    int capacity;
    int size;
} MinHeap;

MinHeap *createMinHeap(int capacity) {
    MinHeap *minHeap = (MinHeap *)malloc(sizeof(MinHeap));
    minHeap->capacity = capacity;
    minHeap->size = 0;
    minHeap->array = (Edge *)malloc(capacity * sizeof(Edge));
    return minHeap;
}

void swapEdge(Edge *a, Edge *b) {
    Edge temp = *a;
    *a = *b;
    *b = temp;
}

void minHeapify(MinHeap *minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size &&
        minHeap->array[left].weight < minHeap->array[smallest].weight)
        smallest = left;

    if (right < minHeap->size &&
        minHeap->array[right].weight < minHeap->array[smallest].weight)
        smallest = right;

    if (smallest != idx) {
        swapEdge(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

```

```c
bool isEmpty(MinHeap *minHeap) { return minHeap->size == 0; }

Edge extractMin(MinHeap *minHeap) {
    if (isEmpty(minHeap)) {
        Edge emptyEdge = {-1, -1, -1};
        return emptyEdge;
    }

    if (minHeap->size == 1) {
        minHeap->size--;
        return minHeap->array[0];
    }

    Edge root = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    minHeap->size--;
    minHeapify(minHeap, 0);

    return root;
}

void insertMinHeap(MinHeap *minHeap, int u, int v, int weight) {
    if (minHeap->size == minHeap->capacity)
        return; // heap full

    minHeap->size++;
    int i = minHeap->size - 1;

    // insert at the bottom and bubble up
    minHeap->array[i].u = u;
    minHeap->array[i].v = v;
    minHeap->array[i].weight = weight;

    while (i != 0 &&
           minHeap->array[(i - 1) / 2].weight > minHeap->array[i].weight) {
        swapEdge(&minHeap->array[i], &minHeap->array[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

void freeMinHeap(MinHeap *minHeap) {
    free(minHeap->array);
    free(minHeap);
}

#endif
```

Graph.h

```c
#ifndef ADJACENCY_LIST_H
#define ADJACENCY_LIST_H

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    struct Node *next;
    int vertex;
    int weight;
} Node;

Node **createAdjacencyList(int vertices) {
    Node **list = (Node **)malloc(vertices * sizeof(Node *));
    for (int i = 0; i < vertices; i++) {
        list[i] = NULL;
    }
    return list;
}

bool addEdgeAdjacencyListDirectedWithoutWeight(Node **list, int u, int v) {
    if (list == NULL)
        return false;
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = 0;
    newNode->next = list[u];
    list[u] = newNode;

    return true;
}

bool addEdgeAdjacencyListUnDirectedWithoutWeight(Node **list, int u, int v) {
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, u, v))
        return false;
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, v, u))
        return false;
    return true;
}

bool addEdgeAdjacencyListDirectedWithWeight(Node **list, int u, int v,
                                            int weight) {
    if (list == NULL)
        return false;

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = weight;
    newNode->next = list[u];
    list[u] = newNode;

```

```
    return true;
}

bool addEdgeAdjacencyListUnDirectedWithWeight(Node **list, int u, int v,
                                              int weight) {
    if (!addEdgeAdjacencyListDirectedWithWeight(list, u, v, weight))
        return false;
    if (!addEdgeAdjacencyListDirectedWithWeight(list, v, u, weight))
        return false;
    return true;
}

void displayAdjacencyList(Node **list, int vertices) {
    printf("\nAdjacency List Representation\n");
    printf("------------------------------\n");
    for (int i = 0; i < vertices; i++) {
        printf("Vertex [%d]: ", i);
        Node *temp = list[i];

        while (temp != NULL) {
            printf("%d(w:%d) -> ", temp->vertex, temp->weight);
            temp = temp->next;
        }
        printf("NULL\n");
    }
    printf("------------------------------------\n");
}

void freeAdjacencyList(Node **list, int vertices) {
    if (list == NULL)
        return;

    for (int i = 0; i < vertices; i++) {
        Node *current = list[i];
        while (current != NULL) {
            Node *temp = current;
            current = current->next;
            free(temp);
        }
    }
    free(list);
}

#endif // ADJACENCY_LIST_H
```